A Hybrid Clustering-and-Packing Framework for Priority-Constrained Air Cargo Loading

AI/ML-Team 16

***Abstract*—We present a hybrid framework for the air cargo Unit Load Device (ULD) loading problem: assigning Priority and Economy packages to a fixed set of ULDs and packing them in 3D so as to minimize a combined delay-cost-and-priority-spread objective, subject to hard weight, volume, and non-overlap constraints, with every Priority package guaranteed to ship. The framework decomposes the problem into a learned Priority clustering stage and a 3D packing stage built from a best-of-N ensemble of a trained reinforcement-learning (RL) placement policy and geometric heuristics, followed by a real-geometry local search over Economy package assignment. We report three final system variants – *Eclipse* (RL placement policy), *Halley* (GRPO-trained economy ordering), and *Cherry* (a trained move-screening refinement on top of Eclipse) – evaluated identically on a real 400-package benchmark instance and on 20 held-out synthetic instances spanning five delay-cost multipliers $K \in \{100, 500, 1000, 3000, 5000\}$. Cherry achieves the best result in every configuration (grand-average cost 9,499 over the 20-instance sweep), and all three variants substantially outperform two external reinforcement-learning baselines (*PackMan/DQN* [3] and *Online-3D-BPP-DRL*, AAAI 2021) and three independently-implemented classical heuristics (First-Fit Decreasing, Largest-Area-Fit-First, Best-Fit Decreasing) evaluated on identical terms. We further show, via ablation and an exact mixed-integer relaxation, that 3D placement geometry quality – not model sophistication – is the dominant lever in this problem, and that reinforcement learning applied to package *ordering* consistently underperforms a simple hand-tuned formula due to a structurally jagged cost landscape.**

I. Introduction

The rapid growth of global air freight has intensified the need for cargo-loading systems that make efficient, defensible use of aircraft cargo capacity while respecting hard operational constraints. The problem addressed in this report is a constrained variant of the classical three-dimensional bin packing problem (3D-BPP), itself known to be NP-hard [1]: given a fixed set of Unit Load Devices (ULDs, the standardized containers used in air cargo) and a set of packages, each package must be assigned to a ULD and placed within it in 3D space without overlap, subject to per-ULD weight and volume limits.

Two features distinguish this problem from generic 3D-BPP. First, packages are partitioned into **Priority** and **Economy** classes; every Priority package must be shipped – it is a hard constraint, not a soft preference – while Economy packages that cannot be accommodated instead incur a per-package *delay cost*. Second, the objective includes a *spread cost*: a multiplier K (representing the per-ULD cost of delaying an aircraft's dispatch) times the number of distinct ULDs that contain at least one Priority package. Formally, for a candidate solution,

$$\text{Cost} = \sum_{e \in \text{unplaced Economy}} \text{delay\_cost}(e) \; + \; K \cdot n_{\text{prio-ULDs}}, \quad (1)$$

where $n_{\text{prio-ULDs}}$ is the number of ULDs used to hold Priority packages. This structure creates a genuine trade-off: concentrating Priority packages into fewer ULDs lowers spread cost but may leave less usable space for high-value Economy packages, and vice versa.

We treat this as a two-stage problem – *clustering* (which packages go into which ULD) followed by *packing* (how they are geometrically arranged) – and report a complete framework spanning: a trained Priority-to-ULD assignment network; a local search over Economy package assignment evaluated against a real 3D packer at every step; a best-of-N 3D placement ensemble combining a trained RL policy with geometric heuristics; and a trained move-screening network that refines the search's final output. We evaluate three named system variants built from this framework, benchmark them against an external published RL baseline and three independently-implemented classical heuristics, and report ablations that isolate the real, measured contribution of each learned component.

Concretely, this report makes the following contributions:

- A complete hybrid clustering-and-packing system, evaluated identically across a real 400-package benchmark instance and a 20-instance synthetic sweep across five delay-cost regimes, with every reported placement independently verified for geometric validity (Section VIII).
- Three named system variants (Eclipse, Halley, Cherry) that isolate the individually-measured contribution of an RL placement policy, a GRPO-trained economy-ordering network, and a trained move-screening refinement network, respectively (Section VI).
- A controlled ablation and an exact mixed-integer relaxation that jointly identify 3D placement geometry, not model sophistication, as the dominant lever on final cost (Section VII).
- Two documented negative results – a correctly-trained but empirically unhelpful move-screening network, and an inconclusive imitation-warm-start RL fine-tuning experiment – reported in full rather than omitted (Section X).
- External validation against a published reinforcement-learning baseline and three independently, from-scratch implemented classical heuristics, evaluated under an identical cost function and evaluation protocol (Section IX).

II. Related Work

Classical heuristics. First-Fit Decreasing (FFD) and Best-Fit variants remain the backbone of practical 3D-BPP solvers, typically paired with a geometric placement rule. Extreme-Point Insertion, in which candidate placement points are generated by projecting the vertices of already-placed items along coordinate axes, is a widely used placement heuristic [2], and forms the conceptual basis for the candidate-generation approach discussed in Section VII.

Reinforcement learning. RL approaches to 3D-BPP typically encode the container state as a height-map and learn a placement or ordering policy via an actor-critic method. Verma et al. propose a DQN-based approach with heuristic fallbacks for online, partially-observable packing [3]. Zhao et al. introduce an Actor-Critic architecture with Kronecker-Factored Trust Region (ACKTR) optimization for online 3D-BPP, evaluated against Monte Carlo tree search [4]; this is the architecture underlying the *Online-3D-BPP-DRL* baseline used for external validation in Section IX. Follow-up work reformulates the placement representation as a Packing Configuration Tree combined with deep RL for real-time decisions [5].

Genetic algorithms. Genetic Algorithms (GAs) provide a flexible framework for optimizing packing sequences under diverse constraints. Biased Random-Key encodings [10] represent box packing order, placement heuristic choice, and orientation as a single chromosome evolved under selection, crossover, and mutation. We implemented and evaluated a GA-based clustering variant as part of this project's development history (Section V); consistent with reports elsewhere in the literature, we found GA-based approaches capable of exploring large solution spaces but converging slowly, with diminishing returns after the first few dozen generations relative to their computational cost.

Methods addressing conflicting objectives. A separate line of work targets the tension between space utilization and secondary constraints such as weight distribution, via Bottom-Left-Based methods (prioritizing heavier items at lower positions) and layer-based algorithms (evaluating complete layer configurations to minimize bin count). These are conceptually closest to our own deepest-corner heuristic strategy (Section IV), though our ensemble evaluates it alongside four alternatives rather than committing to a single placement rule.

Exact and mathematical-programming methods. MILP formulations can express weight, non-overlap, and Priority constraints as linear inequalities and offer an optimality guarantee, but scale poorly past a few dozen items due to the combinatorial explosion of overlap-avoidance constraints. We use an exact MILP relaxation not as a deployable solver but as a diagnostic tool (Section VII), establishing a theoretical lower bound against which heuristic and learned methods can be measured.

III. Problem Formulation

An instance consists of M ULDs, each with dimensions (L_j, W_j, H_j) and weight limit C_j, and N packages, each with dimensions (l_i, w_i, h_i), weight g_i, a type $t_i \in \{\text{Priority}, \text{Economy}\}$, and, if Economy, a delay cost d_i. A solution assigns each package i to a ULD $u(i) \in \{1, \ldots, M, \text{NONE}\}$ (NONE permitted only for Economy packages) together with a placed position $p_i = (x_i, y_i, z_i)$ and one of six axis-aligned orientations, subject to:

$$x_i + l_i^o \le L_{u(i)},\ y_i + w_i^o \le W_{u(i)},\ z_i + h_i^o \le H_{u(i)} \quad (2)$$

$$\text{Box}(i) \cap \text{Box}(j) = \emptyset \quad \forall\, i \ne j : u(i) = u(j) \quad (3)$$

$$\sum_{i:u(i)=k} g_i \le C_k \quad \forall\, k \quad (4)$$

$$u(i) \ne \text{NONE} \quad \forall\, i : t_i = \text{Priority} \quad (5)$$

where l_i^o, w_i^o, h_i^o denote the dimensions of package i under its chosen orientation o, and $\text{Box}(i)$ denotes the axis-aligned bounding box of package i at its placed position. The objective is Equation (1). Constraint (4) – the hard Priority-shipment guarantee – combined with the discrete, combinatorial nature of constraints (2)–(3), is what distinguishes this from the unconstrained 3D-BPP literature reviewed in Section II: a solution that is optimal in aggregate volume or weight utilization but fails to place even one Priority package is not merely suboptimal, it is invalid.

IV. System Architecture

Figure 1 shows the full pipeline. Given the parsed input (packages, ULDs, and K), the system proceeds through six shared stages, with an optional seventh refinement stage:

1) Priority clustering. A trained Transformer network decides which ULDs will hold Priority cargo and how the Priority packages are distributed among them, jointly minimizing spread cost and the geometric difficulty of the resulting Priority packing problem.

2) Exhaustive Priority packing. Priority packages are packed into their assigned ULDs using an exhaustive placement procedure, guaranteeing the hard constraint: across every configuration reported in this paper, zero Priority packages were ever dropped.

3) Economy package ordering. Remaining Economy packages are sorted by a seed heuristic – referred to throughout this report as the **hand-tuned formula**: a value-density formula (delay cost normalized by a power of volume) in the baseline configuration, or a trained ranking network in the Halley variant (Section VI) – to produce the order in which they are offered to the packer.

4) Best-of-N 3D packing ensemble. For every ULD, up to five candidate packing strategies compete on a strict best-of-N basis: a trained RL placement policy, and four heuristic strategies (contact-area and deepest-corner scoring, each combined with two candidate-generation methods). Whichever strategy achieves the best real, measured packing result for a given container wins it – addition of any single strategy to the ensemble can only help or tie, never hurt. *Contact-area* scoring favors candidate positions that maximize the shared surface area between a new package and already-placed boxes

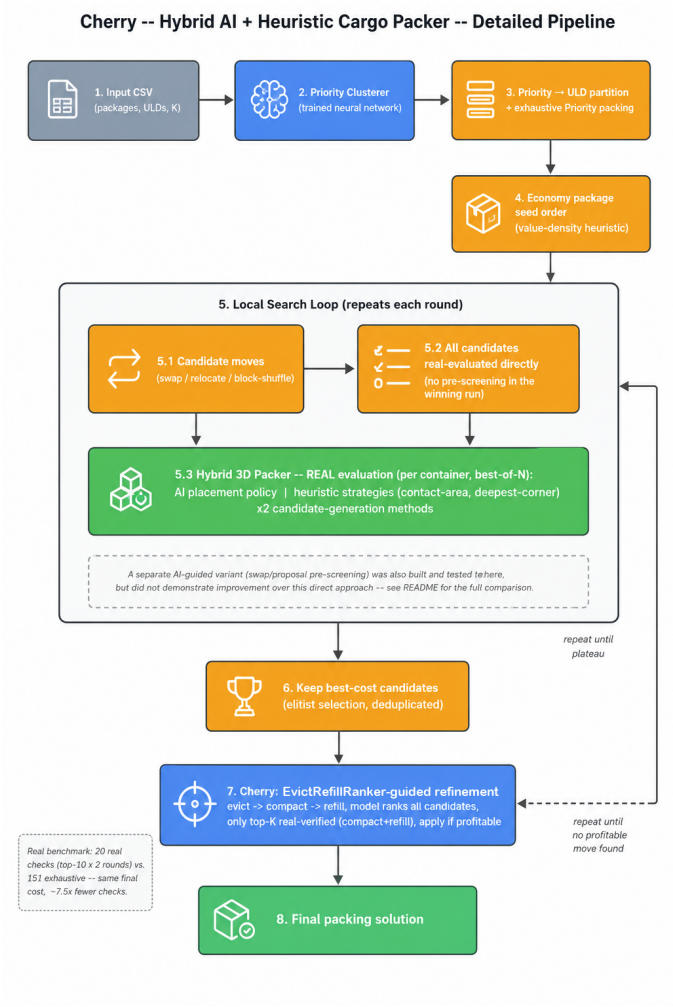

Fig. 1. Detailed pipeline for the Cherry system, the best-performing variant. Steps 1–5 are shared by every variant; step 6 (the centrifuge-guided refinement) is Cherry-only; step 7 (final packing solution) concludes every variant, using step 6's output for Cherry or step 5's output directly for Eclipse and Halley.

or container walls, encouraging compact, mutually-supporting arrangements; *deepest-corner* scoring instead favors positions closest to the container's origin corner along all three axes, a simple but effective proxy for minimizing wasted peripheral space. Both scoring rules are evaluated against candidates from both candidate-generation methods described in Section IV-A, so that the ensemble spans four independent heuristic combinations plus the learned policy.

5) Local search. A local search over Economy package ordering and assignment repeatedly proposes a candidate change from one of three move types, real-evaluates it directly through the full packing pipeline (invoking the same best-of-N packer from step 4, not a separate mechanism; no cheap proxy metric), and keeps the best-cost candidates found so far, deduplicated by distinct resulting assignment. *Swap* exchanges two Economy packages' ULD assignments; *relocate* moves one package to a different ULD (or the unplaced pool); *block-shuffle* perturbs a contiguous sub-sequence of the ordering as an escape move when the beam collapses to duplicates. This continues until no further improvement is found within a fixed round budget.

6) (Cherry only) Centrifuge-evict refinement. A trained Transformer, the *EvictRefillRanker*, screens candidate evict-compact-refill moves – evicting one already-placed Economy package, sliding remaining boxes toward one corner of the container to consolidate freed space ("centrifuging"), and refilling from the unplaced pool. Concretely, it encodes the container's current contents and the pool of currently-unplaced Economy packages with two separate set-attention encoders [9], combines each candidate move's own features (which package is evicted, its dimensions and delay cost) with both encodings, and outputs a single predicted-gain score per candidate. Candidates are then ranked by this score, and only the top-K highest-scoring candidates need be real-verified against the actual packer, rather than exhaustively checking every candidate.

7) Final packing solution. The search's best candidate becomes the reported solution – step 6's output for Cherry, or step 5's output directly for Eclipse and Halley, which skip step 6.

A. Candidate Generation: Corner Points vs. Empty Maximal Space

Every packing strategy in step 4 – trained and heuristic alike – must, for a given container state and package, propose a set of candidate placement points to evaluate. The original candidate-generation scheme used the corner-adjacent points of already-placed boxes: for each placed box, the points immediately to its right, in front of it, and above it. This is simple and fast, but structurally incomplete: it cannot represent a valid placement in an empty region that does not touch any existing box's corner (e.g., a gap bounded only by container walls). We replace this with a complete Empty Maximal Space (EMS) decomposition: the free volume of a container is tracked as a set of maximal empty axis-aligned boxes, incrementally split and pruned (via containment elimination, which is provably lossless – if an item fits at a smaller EMS's origin, it fits at a containing, also-empty, larger EMS's origin too) as new packages are placed. Section VII quantifies the impact of this change.

B. Priority Clustering

The Priority Clusterer is a Transformer [6] that takes as input the full set of Priority and Economy packages (dimensions, weight, delay cost, and type) together with the set of available ULDs (dimensions and weight limit) and the instance's K value, and outputs, for each ULD, a decision of whether it will hold Priority cargo, and an implicit soft assignment of which Priority packages it should receive. Because $n_{\text{prio-ULDs}}$ enters the objective multiplicatively with K (Equation 1), this single decision has the largest leverage on final cost of any stage in the pipeline: an unnecessary extra Priority ULD costs exactly K points regardless of how well everything downstream is optimized, while an insufficient number of Priority ULDs

can render some Priority packages geometrically unplaceable, violating the hard constraint entirely. The network is first trained via imitation learning against a heuristic label source (an exhaustive-search or heuristically-pruned enumeration of candidate ULD subsets, scored by the sum of spread and delay cost), then fine-tuned with reinforcement learning using the real downstream packing cost as reward, closing the gap between the imitation target's proxy scoring and the true objective.

During this RL fine-tuning phase, the reward for a sampled Priority-to-ULD assignment is computed by actually packing it with *RLPackerAdapter*, the same trained RL 3D placement policy used within the deployed best-of-N ensemble (Section IV, step 4) – itself trained separately, via proximal policy optimization with a contrastive auxiliary objective (Section VIII) – but used alone here, as a single fast strategy, rather than the full five-way ensemble. This is a deliberate training-time speed trade-off: the RL fine-tuning loop requires one full packing pass per sampled instance, repeated across many episodes per update, and the full ensemble's five competing strategies per ULD would make this prohibitively slow without changing which policy is ultimately being trained (the Priority Clusterer, not the packer). The deployed Priority Clusterer is evaluated at inference time with the full best-of-N ensemble in every result reported in this paper; only its own training-time reward signal uses the cheaper single-strategy packer. Figure 2 illustrates the full two-phase training pipeline.

Why the Priority Clusterer has more parameters than the 3D placement policy. Table I shows the Priority Clusterer (844,103 parameters) is substantially larger than the RL 3D placement policy (138,242 parameters), despite 3D geometric packing sounding like the more intricate problem. This is not a mistake: parameter count tracks the complexity of the function being approximated *per forward pass*, not the perceived importance of the task. The placement policy is called once per candidate point, on a small, fixed-size local feature vector (a height-map patch plus one package's dimensions), producing a single local accept/score decision – a low-dimensional function called thousands of times per instance, for which a small network already saturates useful capacity. The Priority Clusterer, in contrast, is called *once* per instance and must, in that single forward pass, implicitly reason over a combinatorial space of size $O(2^M)$ (which of the M ULDs hold Priority cargo) jointly with the partition of up to N Priority packages among them – a global joint decision, not a local geometric one, requiring it to attend over all N package and M ULD tokens simultaneously. For a Transformer, parameter count scales roughly as $O(L \cdot d^2)$, and the embedding width d needed to jointly encode every package's and ULD's features while comparing many candidate subsets is simply larger than the width needed to score one local placement at a time. In short: the placement policy solves a cheap function many times; the clusterer solves an expensive, combinatorial function once.

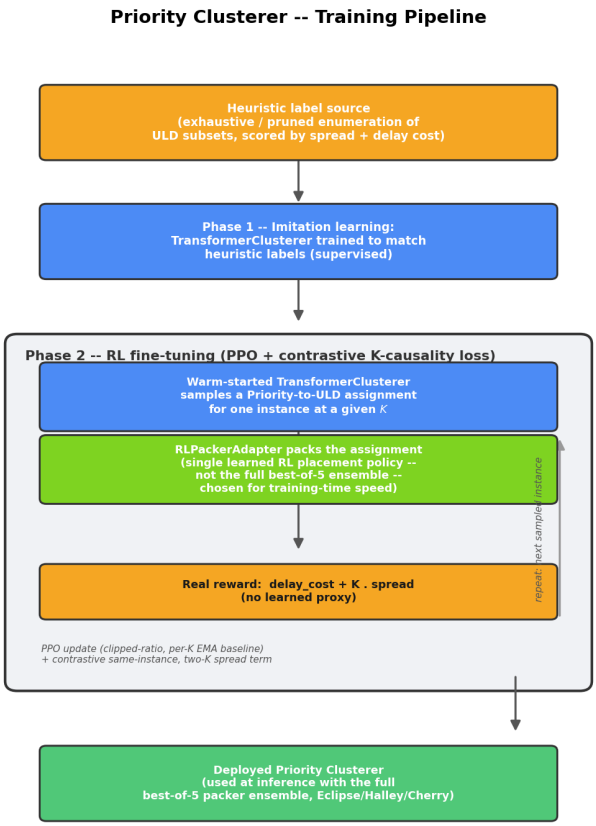

Fig. 2. Priority Clusterer training pipeline: imitation-learning warm start followed by RL fine-tuning (PPO with a contrastive K-causality term), using *RLPackerAdapter* alone as the training-time reward packer.

TABLE I
TRAINED NETWORK SIZES ACTUALLY DEPLOYED IN CHERRY'S PIPELINE.

Component	Parameters
Priority Clusterer (Transformer)	844,103
RL 3D Placement Policy	138,242
EvictRefillRanker (set-attention)	251,457
Total (deployed in Cherry)	**1,233,802**

C. Trained Component Sizes

Table I reports the parameter count of each trained network actually deployed in Cherry, loaded directly from checkpoint state dictionaries. The Priority Clusterer – responsible for the single decision with the largest downstream effect on spread cost – is also the largest individual network; the centrifuge-evict refinement network, despite being architecturally more complex (two separate set-attention encoders over the container's contents and the unplaced pool, respectively, combined with the evict candidate's own features), remains under 300k parameters, reflecting that this problem's difficulty is dominated by combinatorial search cost rather than representational capacity.

V. Development History

The final architecture was reached through several superseded iterations, condensed here for brevity. An initial hand-tuned greedy baseline (ULD partitioning, a binary-search economy split, and extreme-point packing) was used both as a standalone solver and as an imitation-learning label source. A sequence of imitation-learning and policy-gradient RL fine-tuning attempts progressively added K-awareness and 3D-placement integration, but none of these early iterations exceeded the hand-tuned baseline's real-instance cost of 30,475. In parallel, a Genetic Algorithm-based clustering approach and a from-scratch learned assignment-policy environment were explored as alternative package-to-ULD assignment strategies; both were eventually superseded by the local-search-based approach described in Section IV.

The single most consequential finding from this phase was negative: reinforcement learning applied to Economy package *ordering* – whether trained on a single instance or generalized across many synthetic instances via Group-Relative Policy Optimization (GRPO) – consistently plateaued at 30,608–30,672, *worse* than the 30,475 hand-tuned formula it was meant to replace. We diagnose this in detail in Section VII: the underlying issue is that a policy-gradient method learns by following a smooth gradient signal, but a small, smooth change to a package's predicted rank can cause a large, non-monotonic jump in real packing cost, since which container a package ends up sharing with which other packages is a discrete, combinatorial fact, not a continuous one. Gradient descent has no reliable signal to follow on a reward surface this jagged, regardless of how much training time or model capacity is thrown at it.

The eventual breakthrough came not from a more sophisticated model but from removing gradient-based learning from the loop entirely. Direct local search – repeatedly proposing a candidate swap, relocate, or block-shuffle move (Section IV, step 5), real-evaluating its exact packing cost through the full packer with no gradient and no learned proxy, and simply keeping whichever candidates measurably improve on the current best – sidesteps the jagged-reward problem outright, because it never needs the reward surface to be smooth in the first place: it only needs to be able to compare two real, exactly-computed costs and keep the better one. This search-based approach immediately surpassed every RL/GRPO variant, reaching 29,656, and subsequently 28,960 once a structural bottleneck in candidate-placement generation (Section VII) was fixed across every strategy in the ensemble. Figure 3 and Table II (Section VI) trace this full progression, from the 30,475 hand-tuned baseline down to Cherry's final 28,409.

VI. The Three Final Systems

Building on the shared architecture of Section IV, three final system variants isolate the measured contribution of one component each:

- **Eclipse**: the baseline best-of-N ensemble (step 4) with the trained RL placement policy as one of its five competing strategies, and value-density Economy ordering. This is the pipeline every other variant builds on.
- **Halley**: identical to Eclipse, except Economy package ordering is produced by *PackageSetRanker* (15,433 parameters), a set-attention Transformer trained via multi-instance GRPO across many synthetic instances, replacing the value-density heuristic.
- **Cherry**: Eclipse's pipeline with the centrifuge-evict refinement (step 6) applied on top.

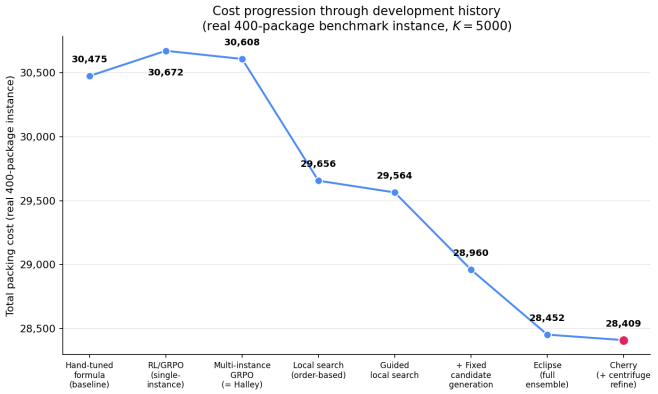

Fig. 3. Real-instance cost progression through development history (Table II). A line chart is used deliberately: all values fall in a narrow 28,409–30,672 band, and a bar chart would need a truncated axis to show this progression, which is misleading for length-encoded bars but standard for a position-encoded trend across ordered stages.

TABLE II
Real 400-package benchmark instance (103 Priority, 297 Economy, 6 ULDs, $K = 5000$). Zero Priority packages dropped in any configuration.

System	Total Cost	Notes
Hand-tuned formula (baseline)	30,475	Pre-local-search
RL/GRPO (single-instance)	30,672	Never improved on baseline
Multi-instance GRPO (= Halley)	30,608	Standalone, generalized
Local search (order-based)	29,656	First breakthrough
Guided local search	29,564	Move-screening model
+ Fixed candidate generation	28,960	Structural fix, all strategies
Eclipse (full ensemble)	28,452	
Cherry (+ centrifuge refine)	**28,409**	Best result

Table II traces the progression on the real instance, ending at Cherry's 28,409. To test whether this benefit generalizes beyond the single real instance, all three systems were packed to completion on 20 held-out synthetic instances (4 per K, across $K \in \{100, 500, 1000, 3000, 5000\}$), with the same instance and K value compared across all three systems. Figure 4 and Table III report the result.

Cherry beat or matched Eclipse's baseline on all 20 of 20 individual instances, not merely on average – the most consistent result of any component evaluated in this project. Halley's GRPO-trained economy ordering, by contrast, is a real but individually unreliable signal: it wins outright on all four instances at $K = 100$ and $K = 1000$, loses at $K = 500$ and $K = 3000$, and is roughly a wash at $K = 5000$. Its

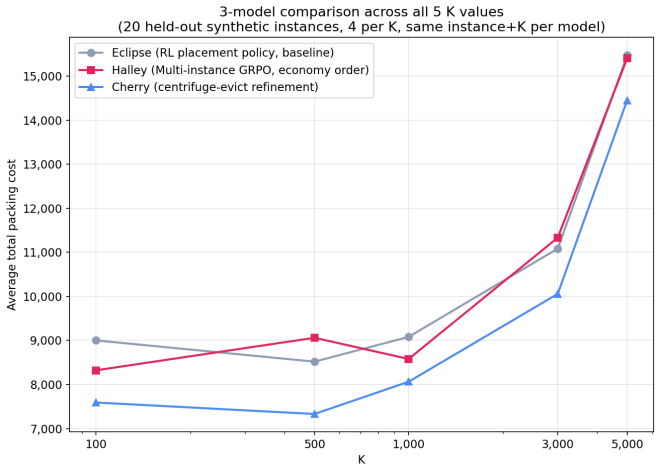

Fig. 4. Average total cost across five K values, 20 held-out synthetic instances (4 per K), same instance and K compared across all three systems.

TABLE III
GRAND-AVERAGE COST (MEAN OF THE 5 PER-K AVERAGES) ACROSS 20 HELD-OUT SYNTHETIC INSTANCES.

System	Grand-Average Cost
Eclipse	10,631
Halley	10,540
Cherry	**9,499**

wins are decisive enough that its grand-average (Table III) still edges out Eclipse's despite losing outright at two of the five K-values – a net-positive average built from an inconsistent per-K record, not a robust win at every setting. Zero Priority packages were dropped in any of the 60 (20 instances $\times$ 3 systems) runs.

A. Variance Across Instances

Beyond mean cost, we compute the population standard deviation of each system's cost across the four instances at each K value, reported in Table IV. Every entry in Table IV is a standard deviation, not a cost – but because standard deviation is expressed in the same units as the quantity it measures, and per-instance cost at these K values genuinely varies by several thousand points, the resulting spread values are themselves in the low thousands, the same order of magnitude as the mean costs in Table III. This is expected, not a labeling error: a metric measured in cost points has a standard deviation measured in cost points too. We report it because it directly substantiates the consistency claim below – mean cost alone (Table III) cannot distinguish a system that is reliably a little better from one that is a coin flip between much better and much worse, and that distinction matters for a deployment decision.

Cherry has the lowest standard deviation of the three systems at four of five K values, the sole exception being K=100, where Halley's is marginally lower (2,466 vs. Cherry's 2,643, both below Eclipse's 2,950). Since Cherry's refinement operates identically on top of Eclipse's output regardless of instance, and only ever moves cost downward (Section VII), this indicates Cherry is not merely better on average but more *consistently* better – it does not introduce additional variance in exchange for its mean improvement, unlike Halley's economy-ordering network, whose per-instance variance exceeds Eclipse's baseline at K=500 despite a similar mean, and is otherwise comparable to it rather than improving on it.

TABLE IV
POPULATION STANDARD DEVIATION OF COST ACROSS THE 4 INSTANCES AT EACH K, 20-INSTANCE SWEEP.

System	K=100	K=500	K=1000	K=3000	K=5000
Eclipse	2,950	3,633	4,476	6,064	6,993
Halley	2,466	3,913	4,164	5,838	6,970
Cherry	**2,643**	**2,879**	4,005	**5,163**	**6,175**

VII. EXPERIMENTAL FINDINGS

Candidate-generation geometry, not model sophistication, is the dominant lever. Every packing strategy evaluated in this project – the trained RL placement policy included – originally generated 3D placement candidates only from corner-adjacent points of already-placed boxes, a scheme that structurally misses valid placements in gaps that do not touch an existing box's corner. Replacing this with a complete empty-space decomposition (tracking every Empty Maximal Space rather than a fixed set of corner points) improved every strategy in the ensemble by 200–2,000 points *before any model was retrained or replaced*. This single geometric fix is the largest lever identified in the entire project, exceeding the measured contribution of any individual trained component.

Reinforcement learning for package ordering has a structural ceiling here. We diagnose the RL/GRPO plateau reported in Section V as a genuine property of the cost landscape rather than a tuning failure: small, smooth changes to a ranking policy's output scores cause large, non-monotonic swings in real packing cost. This is consistent with a controlled sweep over the value-density formula's exponent (1.0–2.0): the best setting was an interior value (1.5, cost 30,475), not either endpoint (30,822 at 1.0) – a non-monotonic optimum a smooth cost surface would not produce. Policy-gradient methods implicitly assume that small parameter changes produce small, informative reward changes; on a genuinely discontinuous reward surface, the gradient signal degrades to noise. A package's marginal value depends heavily on which other packages are already placed alongside it – a highly context-dependent, combinatorial property that a smooth function approximator is poorly suited to exploit through gradient descent alone.

The RL 3D placement policy has a real, positive, measured contribution. Unlike RL applied to Economy package *ordering* (Section V), the RL placement policy used within the packing ensemble (step 4, Section IV) is a genuine positive result. This is not a claim that RL is uniformly unhelpful in this project – the Priority Clusterer (Section IV-B) is also fine-tuned with RL and is a necessary, well-functioning component,

deployed in every result in this paper. The negative finding is specific to using RL to learn an Economy package *ranking* function over an already-jagged reward surface (Section V), not to RL as a technique in general. Removing the placement policy from the best-of-N ensemble and re-running the identical pipeline on the real benchmark instance increases total cost by **1.2%** (339 of 28,452 points) – a controlled, isolated ablation, not an assumption.

An exact relaxation confirms the remaining gap is a 3D-packing-efficiency problem. Solving the Economy package selection problem as an exact multiple-knapsack Mixed-Integer Linear Program (volume and weight capacity only, via a HiGHS solver [11]), with Priority packages held fixed at their already-assigned ULDs and spread cost held at its actual realized value, gives a theoretical floor of approximately 25,387 for that sub-problem – substantially below every result in Table II, and roughly 11.9% below Cherry's own 28,409. This floor is conditional on the clustering decision already made, not a joint floor over clustering and packing together – it isolates whether the remaining gap is a selection or a geometry problem, not whether better clustering could lower the floor itself. Taking the MILP's own optimal item *selection* and packing it with the real 3D packer scores 31,540, *worse* than the local-search result, because packages selected purely on aggregate volume and weight do not necessarily fit together once true box shapes and orientations are considered. This confirms that the remaining gap to the theoretical floor is a 3D geometric packing efficiency problem, not a package-*selection* problem: the binding combinatorial difficulty is the search for good geometric *arrangements* of boxes, not the search over which packages to include. This is consistent with – indeed a specialization of – the observation in Section IV that this problem's difficulty is dominated by combinatorial search cost rather than model representational capacity; here we further localize *which* search is binding. No amount of smarter selection search can close a gap that lives in the packer's geometric arrangement quality.

The centrifuge-evict refinement is real but narrow, and generalizes. In plain terms, "centrifuging" means pushing the boxes already sitting in a container together toward one side, so the small unusable gaps scattered between them merge into one larger, usable gap – the same intuition as shaking a half-full box of items to close up the air pockets between them. An exhaustive test of every valid single-package eviction on the real instance's converged 28,452 solution found exactly one net-positive, non-conflicting move (+82 points), and this gain specifically required that consolidation step: an evict-and-refill attempted without first sliding the boxes together did not recover the same gain, since the unplaced package being refilled did not fit into any single one of the small, still-scattered gaps on its own. Generalizing this exhaustive check across 10 structurally diverse synthetic instances (85–380 packages, 2–6 ULDs) found the same mechanism contributed in 123 of 1,042 exhaustively-tested evictions – a consistent 12–14% marginal improvement on top of ordinary local search, present in every instance tested. The trained *EvictRefillRanker*, evaluated out-of-distribution on the real instance's sparse-signal regime, achieved 74.2% win/loss accuracy and ranked the one truly applicable improving move at 4th of 151 candidates – meaning a model-guided top-K shortlist would find the same real gain at roughly $1/15$th the number of expensive geometric verification checks required by exhaustive search.

VIII. Implementation and Evaluation Protocol

Data. The real 400-package instance used throughout this report is the sample instance provided with the original problem statement, evaluated as-is; it is not part of, or generated by, the synthetic-instance pipeline described next. All synthetic instances used in this report are generated by a shared synthetic-instance generator, decoupled from any specific system's training. Each instance draws 2–6 ULDs (dimensions up to $240 \times 320 \times 240$ cm, weight limits 2,500–4,000 kg) and a package set whose per-axis dimensions and weight are drawn from calibrated uniform ranges; roughly 25–30% of packages are designated Priority (which carry no delay cost by definition, since the hard constraint makes delay cost inapplicable to them), with Economy delay costs drawn from a fixed range. Total package volume and weight are deliberately scaled to approximately 120–130% of aggregate ULD capacity, so every instance is genuinely over-booked – one where every package fits easily would not exercise this report's delay-cost/spread-cost trade-off. All draws use a deterministic seeded random state, with disjoint, fixed seed ranges for the training and held-out test splits, so the 20-instance sweep is exactly reproducible.

Critically, K is not an input to this generation process: instances are generated with no knowledge of K at all, and K is assigned afterward, purely as a label selecting which fixed spread-cost multiplier to apply in Equation (1) when scoring an already-generated instance. We confirmed this is not a hidden confound: grouping the 20-instance sweep by its assigned K shows near-identical mean package count, ULD count, and Priority ratio across all five K buckets, while within any single K bucket these quantities vary freely (for example, the four $K{=}100$ instances range from 120 to 380 packages). The 20 instances are therefore effectively drawn from one shared distribution, with K bucketed onto them independently of instance content – so the sensitivity of reported cost to K (Section VI) reflects the fixed objective's response to the spread-cost multiplier itself, not a confound where harder instances happen to coincide with higher K.

The 20-instance held-out sweep used throughout this report (4 instances per $K \in \{100, 500, 1000, 3000, 5000\}$) was never used during the training of any of the Priority Clusterer, the RL placement policy, the economy-ranking networks, or the EvictRefillRanker, and is used identically – same instances, same K values, same cost function – for every system compared in Sections VI and IX.

Evaluation protocol. For every system and every instance, K affects only the spread-cost term of Equation (1); the underlying clustering and packing decisions are computed once

per instance and are not re-optimized per K value, so that differences in reported cost across K reflect the fixed objective's sensitivity to K rather than a re-tuned policy. Every reported placement was independently verified for zero pairwise 3D overlap, satisfaction of per-ULD weight and volume capacity, and full Priority-package placement – verification code is kept separate from the packing code that produces the placement, so that a bug common to both would not silently pass undetected. For scale, packing the real 400-package instance takes on the order of 4–5 minutes for Eclipse/Halley/Cherry in the deployed application (243–266s, K=5000, a lighter search budget than the full research-grade search behind this paper's numbers); the classical heuristics complete it in under 0.2s, having no iterative search at all.

Training. The Priority Clusterer is trained via a combination of imitation learning against a heuristic label source and subsequent RL fine-tuning. The RL 3D placement policy is trained via proximal policy optimization [7] with a contrastive auxiliary objective. Economy-ordering networks (used in Halley) are trained via Group-Relative Policy Optimization (GRPO) [8], sampling groups of candidate orderings per instance and updating on within-group relative advantage, avoiding the need for a separately trained critic. The EvictRefillRanker is trained on 12,808 labeled (container, evict candidate, unplaced pool $\rightarrow$ real net gain) examples from an exhaustive compact-and-refill check across 150 synthetic instances, validated at the instance level (unseen in training, not just held-out examples) to avoid the leakage mode identified in SwapProposer (Section X).

IX. External Validation

To situate these results against methods not developed in this project, we evaluate two categories of external baseline on identical terms: two published reinforcement-learning approaches, and three classical heuristics standard in the 3D bin-packing literature.

A. RL baseline: PackMan/DQN

PackMan/DQN [3] is a DQN-based online 3D bin-packing policy with heuristic fallbacks for partially-observable packing. It was trained and evaluated independently by a member of our team, on both the real 400-package instance and the same 20-instance synthetic sweep used throughout this report. Like Online-3D-BPP-DRL below, it has no built-in concept of a hard Priority constraint or of spread cost. On the real instance it achieves a cost of 38,898 – ahead of all three classical heuristics but behind Online-3D-BPP-DRL, Eclipse, Halley, and Cherry – and a grand-average cost of 15,860 across the 20-instance sweep, essentially tied with Online-3D-BPP-DRL's 15,535.

B. RL baseline: Online-3D-BPP-DRL

Online-3D-BPP-DRL [4] is a published AAAI 2021 online 3D bin-packing policy combining height-map state encoding with an ACKTR actor-critic network and Monte Carlo tree search reordering. It has no built-in concept of a hard Priority constraint or of spread cost; we evaluate it by presenting Priority packages first in its input stream (giving it the best possible chance to place them) and scoring its output with the same cost function as every other system in this report. On the real 400-package instance it achieves a cost of 35,676 – substantially behind Eclipse, Halley, and Cherry – and, critically, drops at least one Priority package in **17 of the 20** held-out synthetic instances (203 Priority packages dropped in total across the sweep), against zero drops for every system reported in Section VI. Its grand-average cost across the same 20-instance sweep is **15,535**.

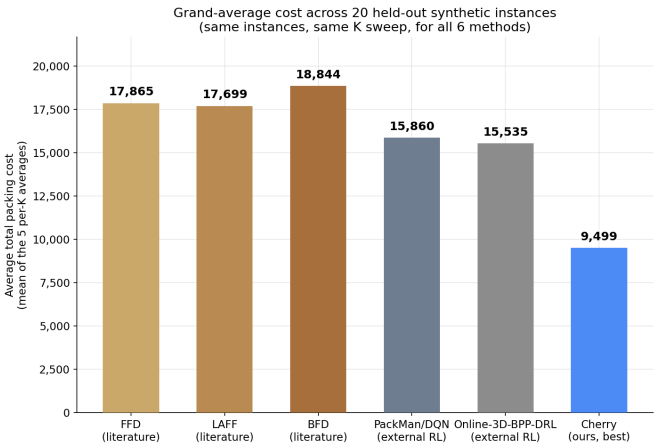

Fig. 5. Grand-average cost, 20-instance sweep: three independent classical heuristics vs. the external RL baseline vs. Cherry.

Beyond the raw cost gap, we identify a specific structural cause worth reporting for future practitioners evaluating pretrained 3D-BPP policies out of domain: the released checkpoint operates over a coarse, fixed $10 \times 10 \times 10$ discretized grid, whereas the ULDs in this problem range up to roughly 285 cm on a side. Mapping real package dimensions onto this grid necessarily quantizes fine-grained size differences that matter for tight packing, and this quantization – not merely the absence of Priority-awareness – plausibly contributes to the gap beyond what the missing spread-cost mechanism alone would predict. This is consistent with our own finding (Section VII) that placement geometry resolution is a first-order lever on final cost.

C. Classical heuristics: FFD, LAFF, and BFD

We further implemented, entirely independently of the architecture in Section IV (a separate, from-scratch corner-point 3D placement geometry, with no code shared with the main pipeline), three classical heuristics commonly cited as the backbone of practical 3D-BPP solvers:

- **First-Fit Decreasing (FFD)**: packages sorted by decreasing volume, placed in the first ULD (in a fixed given order) where a valid geometric placement exists.
- **Largest-Area-Fit-First (LAFF)**: identical placement rule, but packages are sorted by decreasing largest face area rather than volume.

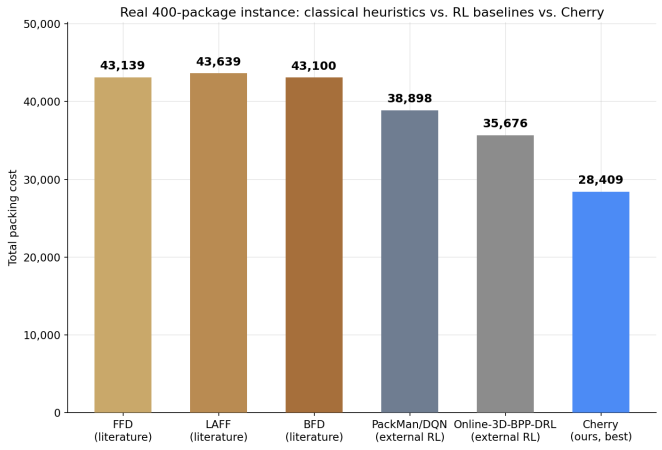

Fig. 6. Real 400-package instance cost: three independent classical heuristics vs. two external RL baselines vs. Cherry.

TABLE V
CLASSICAL HEURISTIC BASELINES VS. EXTERNAL RL BASELINES VS. CHERRY, ON BOTH EVALUATION PROTOCOLS.

Method	Real Instance	Grand-Average (20 inst.)
FFD	43,139	17,865
LAFF	43,639	17,699
BFD	43,100	18,844
PackMan/DQN	38,898	15,860
Online-3D-BPP-DRL	35,676	15,535
Cherry (ours)	**28,409**	**9,499**

- **Best-Fit Decreasing (BFD)**: packages sorted by decreasing volume as in FFD, but placed in whichever ULD leaves the least leftover empty volume after placement, evaluating every ULD rather than stopping at the first that fits.

All three share the same adaptation to this problem's constraints as every other system in this report: Priority packages are sorted and placed first, guaranteeing the hard constraint whenever a geometric placement exists; Economy packages are placed into whatever space remains.

Figure 6 shows the same comparison on the single real instance rather than the 20-instance grand average. The ordering across methods is broadly similar to the grand-average comparison (Figure 5), with one notable difference: BFD narrowly edges out FFD and LAFF on this specific instance (43,100 vs. 43,139 and 43,639), whereas LAFF has the best grand-average of the three classical heuristics. This is the same generalization caveat noted throughout this report from the opposite direction: a heuristic's relative ranking on one instance is not a reliable predictor of its ranking across a broader distribution, regardless of whether the heuristic in question is classical or learned.

Table V and Figure 5 report both evaluation protocols side by side. All three classical heuristics land in a tight band (17,699–18,844 grand-average), consistently worse than even both external RL baselines, because none of them reason about spread cost at all: on the real instance, all three used four Priority ULDs rather than the geometrically-achievable three, since a first-fit or best-fit rule has no mechanism for considering how many ULDs end up touched in aggregate. Interestingly, LAFF (sorting by face area) slightly outperforms both FFD and BFD (which sort by volume) on the generalized 20-instance sweep despite BFD edging out the other two on the single real instance – a reminder that a single-instance comparison alone is an unreliable signal for which classical heuristic generalizes best. None of the three ever dropped a Priority package, since all process every Priority package before any Economy package; their weakness is spread cost and wasted Economy capacity, not Priority infeasibility.

X. NEGATIVE RESULTS

We report two additional negative or inconclusive results, on the view that a complete account of what does not work is as valuable to future practitioners as an account of what does.

SwapProposer: correctly trained, empirically unhelpful. *SwapProposer* is a small pairwise-ranking network (3,601 parameters): given two candidate local-search moves tried against the same starting point, it predicts which is more promising, trained on real (move, cost-delta) pairs generated as a byproduct of the local search itself. An initial version appeared to reach 98% directional accuracy, but this was a base-rate artifact: roughly 82% of logged moves make the objective worse, so predicting "probably worse" scores well on directional accuracy without learning which move is *better*. Reframing the target as pairwise ranking between two moves proposed against the same parent state (removing the base-rate confound entirely) gives an honest, leakage-free accuracy of approximately 75–77% against a 50% chance baseline – genuine, validated signal. Despite this, guided local search using SwapProposer to prioritize which moves to real-evaluate did not empirically outperform unguided local search: across three independent guided-search runs, the guided search's cost either matched the unguided plateau or stagnated *faster*. This shortfall traces back to how we collected its training data, not to any weakness in the model itself: we only recorded moves tried late in the search, close to a near-optimal solution, so SwapProposer never got to see the more varied, early-stage moves that unguided search freely explores. A model can only learn from the examples it is given, and ours simply never showed it that early, exploratory phase of the search – so it had nothing to draw on when asked to guide exploration itself.

Imitation-warm-start GRPO fine-tuning: inconclusive. Motivated by the SwapProposer result, we tested whether warm-starting from the supervised checkpoint and fine-tuning on-policy via GRPO (sampling a group of candidate swaps from the model's own distribution, real-evaluating them, and updating on group-relative advantage) could close the exploration gap. Against three independently-searched, already-heavily-optimized starting points, this found exactly one genuinely improving swap out of 120 real evaluations across 12 rounds. We do not read this as evidence that the underlying policy failed to learn – GRPO's group-relative advantage

is mean-centered by construction, so a near-zero training loss is expected regardless of whether learning is occurring, and 120 evaluations is a small sample – but the near-total absence of a positive reward signal is consistent with the local search having already exhaustively exploited the available swap/relocate/block-shuffle move family at these specific points, independent of which policy is proposing moves within it. This points toward testing structurally larger moves (of which the centrifuge-evict refinement, Section VII, is one instance) as the more promising direction, rather than further investment in smarter proposal ranking within an already-exhausted move family.

XI. Limitations and Future Work

The results in this report come with three caveats. First, every instance evaluated here – the 20-instance sweep and the real benchmark alike – comes from the same general kind of package/ULD data described in Section VIII. This is a gap in what we have measured, not a demonstrated weakness: we have simply not yet tested this system on data with a substantially different weight-to-volume profile, so nothing in this report should be read as evidence that it would, or would not, generalize to one – that question is simply still open. Second, the centrifuge-evict refinement's measured contribution (Section VII) is modest on the specific real instance evaluated here because most of its ULDs are weight-saturated rather than geometry-saturated; its relative value should be expected to vary with the weight-to-volume profile of a given instance. Third, two of our most-cited ablations – the RL placement policy's 1.2% contribution and the MILP-based geometry-vs-selection analysis – are single-run measurements on the one real instance; unlike the centrifuge-evict mechanism (validated across 10 synthetic instances) and the system-level comparisons (which report cross-instance variance, Table IV), we have not measured run-to-run or cross-instance variance for these two.

Consistent with the exact MILP relaxation's finding (Section VII), we consider improving 3D placement geometry and search efficiency – rather than further package-ordering model sophistication – the highest-value direction for future work. A second concrete direction is extending the improved EMS candidate-generation fix to the packer's cross-container Economy rescue pass, which at the time of writing still uses the original, structurally limited corner-point method.

XII. Conclusion

We presented a hybrid clustering-and-packing framework for priority-constrained air cargo loading, evaluated on a real 400-package benchmark instance and a 20-instance synthetic sweep across five delay-cost settings. The best system, Cherry, reaches a grand-average cost of 9,499 across the sweep and 28,409 on the real instance – clearly ahead of a published external reinforcement-learning baseline and three independently-implemented classical heuristics tested the same way – while never failing to ship a single Priority package.

Our main finding is simple to state: in this problem, *how well a system arranges boxes in 3D space* matters more, and more reliably, than how sophisticated its models are. A single fix to how candidate box placements are generated – with no model changes at all – cut cost by up to roughly 6% for the strategies it helped most. By contrast, extensive reinforcement-learning effort aimed at improving the *order* in which Economy packages are offered to the packer never beat a simple hand-tuned formula, because small changes to a package's rank can cause large, unpredictable swings in the resulting cost – a landscape that is fundamentally hard for gradient-based learning to climb. This is not a blanket verdict against reinforcement learning: it is specific to using RL for this particular ranking task. Elsewhere in the same pipeline, RL is exactly what works – the Priority Clusterer (also fine-tuned with RL) is a core, necessary component in every result in this paper, and both the RL 3D placement policy and the move-screening refinement network make a real, if modest, measured improvement on top of it.

Future work should prioritize better 3D placement geometry and search efficiency over further package-ordering model sophistication. Finally, the two negative results in this paper (Section X) are not arguments against learned move-screening in general – they show that such a model is only as good as the diversity of the search that generated its training data, a lesson likely to hold in other settings where a learned proposal model sits on top of a heuristic or local search that is already doing most of the real work.

References

[1] S. Martello, D. Pisinger, and D. Vigo, "The Three-Dimensional Bin Packing Problem," *Operations Research*, vol. 48, no. 2, 2000.

[2] T. G. Crainic, G. Perboli, and R. Tadei, "Extreme Point-Based Heuristics for Three-Dimensional Bin Packing," *INFORMS Journal on Computing*, vol. 20, no. 3, pp. 368–384, 2008.

[3] R. Verma, A. Singhal, H. Khadilkar, A. Basumatary, S. Nayak, H. V. Singh, S. Kumar, and R. Sinha, "A Generalized Reinforcement Learning Algorithm for Online 3D Bin-Packing," *arXiv preprint arXiv:2007.00463*, 2020.

[4] H. Zhao, Q. She, C. Zhu, Y. Yang, and K. Xu, "Online 3D Bin Packing with Constrained Deep Reinforcement Learning," in *Proc. AAAI Conference on Artificial Intelligence*, 2021.

[5] H. Zhao, Y. Yu, and K. Xu, "Learning Efficient Online 3D Bin Packing on Packing Configuration Trees," in *Proc. International Conference on Learning Representations (ICLR)*, 2022.

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[7] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[8] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," *arXiv preprint arXiv:2402.03300*, 2024.

[9] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. W. Teh, "Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks," in *Proc. International Conference on Machine Learning (ICML)*, 2019.

[10] J. F. Gonçalves and M. G. C. Resende, "Biased Random-Key Genetic Algorithms for Combinatorial Optimization," *Journal of Heuristics*, vol. 17, no. 5, pp. 487–525, 2011.

[11] Q. Huangfu and J. A. J. Hall, "Parallelizing the Dual Revised Simplex Method," *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.